.UBAfiuba
FACULTAD DE INGENIERÍA

Teoría de Algoritmos
(75.29) Curso Buchwald - Genender

Trabajo Práctico 2
Programación Dinámica For The Win

6 de noviembre de 2026

Guadalupe Schell
114335

Abril Erin Borsnich
114760

Índice

1. Análisis del algoritmo **3**
1.1. Enunciado del problema . . . 3
1.2. Análisis del problema y algoritmo planteado . . . 3
1.3. Ejemplos de ejecución . . . 4
1.3.1. Ejemplo 1 . . . 4
1.3.2. Ejemplo 2: Sophia no puede ganar . . . 5
1.3.3. Ejemplo 3: estrategia greedy frente a programación dinámica . . . 5

2. Optimalidad del algoritmo **6**
2.1. Demostración . . . 6

3. Implementación del algoritmo **8**
3.1. Código funcional . . . 8
3.2. Complejidad . . . 10
3.3. Análisis de la variabilidad de los valores de las monedas . . . 11
3.4. Medición de tiempos y análisis de complejidad . . . 12

4. Conclusiones **13**

1. Análisis del algoritmo

1.1. Enunciado del problema

Sophia y Mateo juegan a un juego utilizando una fila de n monedas, donde cada moneda tiene asociado un valor. En cada turno solamente puede retirarse una de las dos monedas que se encuentran en los extremos de la fila. Al elegirla, la remueve de la fila, y le toca luego al otro jugador, quien debe elegir otra moneda siguiendo la misma regla. Siguen agarrando monedas hasta que no quede ninguna. Ganará quien acumule el mayor valor total.

Mateo aprendió sobre algoritmos greedy, y aplica esa técnica a la hora de elegir una moneda. En cada turno elige la moneda de mayor valor entre los dos extremos disponibles.

Sophia, en cambio, comenzó a aprender sobre programación dinámica y va a aplicar esta nueva técnica para asegurarse ganar siempre que pueda.

El objetivo es determinar la máxima ganancia que puede obtener Sophia si toma sus decisiones de manera óptima, teniendo en cuenta que Sophia empieza el juego y que Mateo siempre elegirá la moneda más grande para sí entre la primera y la última moneda. Esto no necesariamente le asegurará a Sophia ganar, ya que la estrategia de Mateo puede impedirlo.

1.2. Análisis del problema y algoritmo planteado

Llamamos M el arreglo de monedas, donde $M[i]$ representa el valor de la moneda ubicada en la posición i.

Definimos $OPT(i, j)$ como la máxima ganancia que puede obtener Sophia si le toca jugar y quedan las monedas del intervalo $[i, j]$ suponiendo que Mateo aplica siempre la estrategia Greedy descrita: si $M[i] \geq M[j]$ toma la izquierda, si no, la derecha (ante empate toma la izquierda).

Los intervalos son válidos siempre que $i \leq j$. Si $i > j$, el intervalo está vacío y la ganancia es cero.

En cada estado, Sophia tiene dos alternativas:

- **Tomar la moneda izquierda** $M[i]$**.** Luego de su elección, Mateo debe elegir entre $M[i+1]$ y $M[j]$.
  - Si $M[i+1] \geq M[j]$, Mateo toma la moneda $M[i+1]$ y Sophia jugará su próximo turno con las monedas del intervalo $[i+2, j]$.
  - Si $M[i+1] < M[j]$, Mateo toma la moneda $M[j]$ y Sophia vuelve a jugar con las monedas del intervalo $[i+1, j-1]$.
- **Tomar la moneda derecha** $M[j]$**.** Luego de su elección, Mateo debe elegir entre $M[i]$ y $M[j-1]$.
  - Si $M[i] \geq M[j-1]$, Mateo toma la moneda $M[i]$ y Sophia vuelve a jugar con las monedas del intervalo $[i+1, j-1]$.
  - Si $M[i] < M[j-1]$, Mateo toma la moneda $M[j-1]$ y Sophia jugará su próximo turno con las monedas del intervalo $[i, j-2]$.

Por lo tanto, la recurrencia evalúa cuál de los dos extremos le conviene más a Sophia. Para calcularlo, analiza el intervalo que quedará disponible en su próximo turno, descontando la moneda que Mateo elegirá con su estrategia Greedy antes de que ella vuelva a jugar.

Para todo intervalo con $j - i + 1 \geq 3$, la recurrencia es:

$$OPT(i, j) = \text{máx} \begin{cases} M[i] + A(i, j), \\ M[j] + B(i, j) \end{cases}$$

donde

$$A(i,j) = \begin{cases} OPT(i+2,j), & \text{si } M[i+1] \geq M[j], \\ OPT(i+1,j-1), & \text{si } M[i+1] < M[j], \end{cases}$$

y

$$B(i,j) = \begin{cases} OPT(i+1,j-1), & \text{si } M[i] \geq M[j-1], \\ OPT(i,j-2), & \text{si } M[i] < M[j-1]. \end{cases}$$

Casos base:

- **Si $i > j$:** Entonces $OPT(i,j) = 0$, debido a que ya no quedan monedas en la mesa.
- **Si $i = j$:** Entonces $OPT(i,i) = M[i]$, porque al quedar una sola moneda disponible, Sophia se la lleva de forma directa.
- **Si $i+1 = j$:** Hay dos monedas. Sophia toma la mayor de las dos monedas y Mateo se lleva la restante, por lo que $OPT(i,i+1) = \max\{M[i], M[i+1]\}$. Los intervalos siguientes ($[i+2,j]$ o $[i+1,j-1]$) son vacíos y valen 0.

En la implementación, los subintervalos vacıos se tratan como si tuvieran valor cero. La recurrencia se evalúa para intervalos de longitud creciente: primero los de longitud uno, luego los de longitud dos y ası sucesivamente hasta el intervalo completo [0, n 1]. La respuesta buscada queda en $OPT[0][n-1]$.

Como la decisión de Mateo queda determinada por los extremos disponibles y por su regla greedy, cada alternativa de Sophia conduce a un único subproblema. De esta manera, basta con comparar las dos alternativas y guardar el máximo, sin recalcular repetidamente los mismos subproblemas.

1.3. Ejemplos de ejecución

1.3.1. Ejemplo 1

Consideremos una secuencia de seis monedas:

$$M = [20, 5, 15, 10, 8, 25]$$

La matriz OPT almacena en cada celda la máxima ganancia que puede obtener Sophia cuando le toca jugar y quedan las monedas del intervalo indicado. Los intervalos vacıos no se almacenan en la matriz y se consideran de valor cero.

Evaluación de las dos primeras alternativas. Al evaluar el intervalo completo $[0,5]$, Sophia tiene dos opciones:

- **Tomar 20, el extremo izquierdo.** Queda $[5,15,10,8,25]$. Mateo compara 5 y 25, y toma 25. Sophia vuelve a jugar con $[5,15,10,8]$, cuyo valor es OPT[1][4] = 23. La ganancia total que puede obtener Sophia por esta alternativa es 20 + 23 = 43.
- **Tomar 25, el extremo derecho.** Queda $[20,5,15,10,8]$. Mateo compara 20 y 8, y toma 20. Sophia vuelve a jugar con $[5,15,10,8]$, cuyo valor es OPT[1][4] = 23. La ganancia total de Sophia por esta alternativa es 25 + 23 = 48.

Por lo tanto, la decisión óptima inicial es tomar la moneda derecha, de valor 25. La matriz final es:

$i \setminus j$	0	1	2	3	4	5
0	20	20	25	35	35	**48**
1	–	5	**15**	15	**23**	40
2	–	–	15	15	23	35
3	–	–	–	10	10	33
4	–	–	–	–	8	25
5	–	–	–	–	–	25

Reconstrucción del camino óptimo. La secuencia de decisiones es:

1. El estado inicial es $\mathbf{OPT[0][5] = 48}$. Sophia toma la moneda derecha, de valor **25**. Mateo compara los extremos restantes, 20 y 8, y toma 20.
2. Queda el intervalo $[1, 4]$, correspondiente a $[5, 15, 10, 8]$, con $\mathbf{OPT[1][4] = 23}$. Sophia toma la moneda derecha, de valor **8**. Mateo compara 5 y 10 y toma (**10**).
3. Queda el intervalo $[1, 2]$, correspondiente a $[5, 15]$, con $\mathbf{OPT[1][2] = 15}$. Sophia toma (**15**). Mateo recibe la moneda restante, de valor 5.

Sophia obtiene $25 + 8 + 15 = 48$, mientras que Mateo obtiene $20 + 10 + 5 = 35$. Por lo tanto, en este ejemplo Sophia gana.

1.3.2. Ejemplo 2: Sophia no puede ganar

Consideremos ahora:

$$M = [1, 10, 5]$$

Al evaluar el intervalo completo $[0, 2]$, Sophia compara las dos alternativas:

- **Tomar 1, el extremo izquierdo.** Mateo recibe el arreglo $[10, 5]$ y toma 10. Sophia se queda con la moneda de valor 5. Su ganancia es $1 + 5 = 6$.
- **Tomar 5, el extremo derecho.** Mateo recibe el arreglo $[1, 10]$ y toma 10. Sophia se queda con la moneda de valor 1. Su ganancia es $5 + 1 = 6$.

La matriz final queda:

$i \setminus j$	0	1	2
0	1	10	**6**
1	–	10	**10**
2	–	–	**5**

Ambas alternativas dan una ganancia de 6 para Sophia. Si se desempata eligiendo el extremo izquierdo, Sophia toma 1, Mateo toma 10 y Sophia toma 5. Sophia obtiene 6 y Mateo obtiene 10. Por lo tanto, la ganancia óptima de Sophia es 6, pero no alcanza para ganar. Este ejemplo muestra que el algoritmo maximiza la ganancia de Sophia, aunque las reglas del juego no siempre le permiten superar a Mateo.

1.3.3. Ejemplo 3: estrategia greedy frente a programación dinámica

Consideremos la secuencia

$$M = [1, 2, 1, 1, 1]$$

Este ejemplo permite comparar una estrategia greedy para Sophia con la estrategia óptima calculada mediante programación dinámica. Para que la comparación sea reproducible, suponemos que, ante un empate entre los extremos, tanto Sophia como Mateo eligen el extremo izquierdo.

Si Sophia también juega greedy.

1. Sophia ve 1 y 1 en los extremos y, por el desempate, toma el 1 de la izquierda. Queda $[2, 1, 1, 1]$.
2. Mateo compara 2 y 1 y toma el 2 de la izquierda. Queda $[1, 1, 1]$.
3. Sophia toma el 1 de la izquierda. Queda $[1, 1]$.
4. Mateo toma el 1 de la izquierda y Sophia recibe el último 1.

Sophia suma $1 + 1 + 1 = 3$ y Mateo suma $2 + 1 = 3$. El resultado es un empate.

Si Sophia también juega de forma óptima.

1. Sophia toma el 1 del extremo derecho. Queda $[1, 2, 1, 1]$.
2. Mateo encuentra un empate entre los extremos, ambos de valor 1, y toma el de la izquierda. Queda $[2, 1, 1]$.
3. Sophia toma el 2 de la izquierda. Queda $[1, 1]$.
4. Mateo toma el 1 de la izquierda y Sophia recibe el último 1.

En este caso, Sophia suma $1 + 2 + 1 = 4$ y Mateo suma $1 + 1 = 2$. Sophia gana.

La diferencia se debe a que la primera elección de Sophia no se basa únicamente en el valor inmediato de la moneda. Aunque ambos extremos valen 1, tomar el extremo derecho cambia la respuesta greedy de Mateo: este toma el 1 izquierdo y deja disponible la moneda de valor 2 para el siguiente turno de Sophia. La programación dinámica detecta esta consecuencia futura y obtiene una ganancia de 4, superior a los 3 puntos que consigue la estrategia greedy.

2. Optimalidad del algoritmo

2.1. Demostración

Llamando:

- *jugada*: a la extracción de una moneda de uno de los extremos por parte de un jugador.
- *ronda*: a una jugada de Sophia seguida de una jugada de Mateo.
- *partida desde* $[i, j]$: a la sucesión de rondas que se juegan, comenzando con Sophia, hasta agotar las monedas del intervalo $[i, j]$.

Como en cada turno solo se retira una moneda de un extremo, todo estado del juego es un intervalo contiguo $[i, j]$ de la fila original.

La estrategia de Mateo es una estrategia Greedy:

$$g(i,j) = \begin{cases} \text{izquierda } (M[i]) & \text{si } M[i] \geq M[j], \\ \text{derecha } (M[j]) & \text{si } M[i] < M[j]. \end{cases}$$

(Fijamos que ante empate toma la izquierda.)

Como las respuestas de Mateo son automáticas y predecibles, cada elección de Sophia (tomar la moneda izquierda o tomar la moneda derecha) en cada uno de sus turnos define por completo la partida. Definimos el *valor óptimo real*

$$V(i,j) = \max_{\text{partidas desde } [i,j]} \big(\text{suma de las monedas que toma Sophia}\big), \qquad V(i,j) = 0 \text{ si } i > j.$$

Por otro lado, $OPT(i, j)$ es el valor definido por la recurrencia y los casos base. Lo que queremos probar es que ambos coinciden.

Queremos demostrar que para todo i, j vale $OPT(i, j) = V(i, j)$.

Demostración por inducción sobre la longitud $l = j - i + 1$ del intervalo.

Casos base:

- $l = 0$ $(i > j)$: el intervalo está vacío, por lo que $V(i, j) = 0 = OPT(i, j)$.
- $l = 1$ $(i = j)$: el intervalo contiene una sola moneda. Sophia debe tomar $M[i]$ y la partida termina, por lo que $V(i, i) = M[i] = OPT(i, i)$.
- $l = 2$ $(j = i + 1)$: el intervalo tiene dos monedas. Sophia toma una moneda y Mateo toma la restante, así que Sophia obtiene $V(i, i + 1) = \text{máx}\{M[i], M[i + 1]\} = OPT(i, i + 1)$.

Paso inductivo: Sea $l \geq 3$ y supongamos que $OPT(a, b) = V(a, b)$ para todo intervalo $[a, b]$ de longitud menor que l. Sea $[i, j]$ de longitud l. Como la respuesta de Mateo es una función de los extremos actuales, una partida desde $[i, j]$ queda determinada por la sucesión de elecciones de Sophia. Sea $S(P)$ la suma de las monedas de Sophia en la partida P, y sean $\mathcal{P}_I$ y $\mathcal{P}_D$ las partidas desde $[i, j]$ cuya primera elección de Sophia es la izquierda y la derecha, respectivamente. Toda partida pertenece a exactamente uno de los dos, por lo tanto

$$V(i, j) = \text{máx}\Big\{ \max_{P \in \mathcal{P}_I} S(P), \quad \max_{P \in \mathcal{P}_D} S(P) \Big\}.$$

Afirmamos que

$$\max_{P \in \mathcal{P}_I} S(P) = M[i] + V(I_A),$$

donde $I_A = [i + 2, j]$ si $M[i + 1] \geq M[j]$ e $I_A = [i + 1, j - 1]$ si $M[i + 1] < M[j]$, y análogamente que

$$\max_{P \in \mathcal{P}_D} S(P) = M[j] + V(I_B),$$

donde $I_B = [i + 1, j - 1]$ si $M[i] \geq M[j - 1]$ e $I_B = [i, j - 2]$ si $M[i] < M[j - 1]$. Lo probamos para cada alternativa.

- **Alternativa 1 (Tomar izquierda $M[i]$):** Tras la elección de Sophia, a Mateo le queda disponible el intervalo $[i + 1, j]$ y juega según su regla greedy. Si $M[i + 1] \geq M[j]$, toma $M[i + 1]$ y queda $[i + 2, j]$; si $M[i + 1] < M[j]$, toma $M[j]$ y queda $[i + 1, j - 1]$. En ambos casos, el intervalo con el que Sophia vuelve a jugar es exactamente I_A, de longitud $l - 2$, el que aparece en $A(i, j)$.

  Después de que Sophia toma $M[i]$ y Mateo juega en base a su estrategia greedy, el juego queda exactamente igual que una partida nueva sobre el intervalo I_A: lo único que importa para lo que sigue son las monedas que quedan, no cómo se llegó a ellas. Esto significa que todas las formas de seguir jugando después de esa primera ronda son las mismas que las posibles partidas que se pueden jugar desde I_A. Y también sucede al revés: cualquier partida que empiece desde I_A se puede realizar desde el intervalo original $[i, j]$, si primero Sophia toma $M[i]$ y Mateo responde siguiendo su estrategia greedy.

  En esa primera ronda Sophia suma únicamente $M[i]$ a su puntaje. Todo lo demás que junta Sophia es lo que junta en la partida desde I_A. Por lo tanto, el puntaje total de Sophia es:

  $$\text{puntaje de Sophia} = M[i] + \text{puntaje de Sophia en la partida desde } I_A.$$

  Como $M[i]$ es un valor fijo, para conseguir el mejor puntaje Sophia debe elegir la mejor partida desde I_A. Entonces, el máximo puntaje que puede conseguir si comienza tomando la moneda izquierda es:

  $\boxed{M[i] + V(\mathrm{I}_A)}$

- **Alternativa 2 (Tomar derecha $M[j]$):** De la misma manera, si Sophia toma $M[j]$, a Mateo le queda el intervalo $[i, j-1]$ y juega según su estrategia greedy. Si $M[i] \geq M[j-1]$, toma $M[i]$

y queda $[i+1, j-1]$; si $M[i] < M[j-1]$, toma $M[j-1]$ y queda $[i, j-2]$. El intervalo con el que Sophia vuelve a jugar es exactamente I_B, de longitud $l-2$, el que aparece en $B(i,j)$. Con el mismo razonamiento que en la alternativa anterior, cada partida desde $[i,j]$ que empieza tomando $M[j]$ se corresponde con una partida desde I_B, y el puntaje de Sophia es $M[j]$ más su puntaje en esa partida. Por lo tanto, el máximo en esta alternativa es $M[j] + V(I_B)$.

Los intervalos I_A e I_B tienen longitud $l-2$, que es menor que l (y al menos 1, porque $l \geq 3$). Por lo tanto, la hipótesis inductiva nos asegura que el valor de la recurrencia coincide con el valor real en ellos:

$$V(I_A) = OPT(I_A) = A(i,j) \qquad \text{y} \qquad V(I_B) = OPT(I_B) = B(i,j).$$

Toda partida desde $[i,j]$ empieza con una de las dos alternativas, y el mejor puntaje posible de Sophia es el mayor entre ambas. Reemplazando lo que probamos en cada alternativa, llegamos a:

$$V(i,j) = \text{máx}\{M[i]+V(I_A),\ M[j]+V(I_B)\} = \text{máx}\{M[i]+A(i,j),\ M[j]+B(i,j)\} = OPT(i,j). \qquad \blacksquare$$

Esto prueba que la ecuación de recurrencia es correcta para todos los intervalos. En particular, $OPT(0, n-1) = V(0, n-1)$ es la máxima ganancia que Sophia puede obtener en el juego completo.

3. Implementación del algoritmo

3.1. Código funcional

La implementación utiliza un enfoque iterativo (*Bottom-Up*). Se construye la matriz OPT desde los intervalos más pequeños hasta el intervalo completo.

```python
def resolver(monedas):
    n = len(monedas)
    if n == 0:
        return [], 0, 0


    opt = [[0] * n for _ in range(n)]

    # Caso base queda una sola moneda
    for i in range(n):
        opt[i][i] = monedas[i]

    for largo in range(2, n + 1):
        for i in range(n - largo + 1):
            j = i + largo - 1

            # sophia toma la moneda izquierda
            if monedas[i + 1] >= monedas[j]:
                if i + 2 <= j:
                    futuro_izq = opt[i + 2][j]
                else:
                    futuro_izq = 0
            else:
                if i + 1 <= j - 1:
                    futuro_izq = opt[i + 1][j - 1]
                else:
                    futuro_izq = 0
            izq = monedas[i] + futuro_izq

            # sophia toma la moneda derecha
            if monedas[i] >= monedas[j - 1]:
                if i + 1 <= j - 1:
                    futuro_der = opt[i + 1][j - 1]
                else:
                    futuro_der = 0
            else:
```

```
                if i <= j - 2:
                    futuro_der = opt[i][j - 2]
                else:
                    futuro_der = 0
            der = monedas[j] + futuro_der

            # en caso de empate se elige el extremo izquierdo
            if izq >= der:
                opt[i][j] = izq
            else:
                opt[i][j] = der

    return reconstruir_elecciones(opt, monedas)

def reconstruir_elecciones(opt, monedas):
    n = len(monedas)
    decisiones = []
    i = 0
    j = n - 1

    ganancia_mateo = 0

    while i <= j:
        if i == j:
            # Caso base: queda unicamente esa moneda
            decisiones.append(f"Sophia toma la izquierda ({monedas[i]})")
            break

        # uso alternativa de tomar la izquierda para deducir la decisi n
        if monedas[i + 1] >= monedas[j]:
            if i + 2 <= j:
                futuro_izq = opt[i + 2][j]
            else:
                futuro_izq = 0
        else:
            if i + 1 <= j - 1:
                futuro_izq = opt[i + 1][j - 1]
            else:
                futuro_izq = 0
        valor_si_izq = monedas[i] + futuro_izq

        # Si el valor calculado coincide con la matriz, Sophia eligio la izquierda
        if opt[i][j] == valor_si_izq:
            decisiones.append(f"Sophia toma la izquierda ({monedas[i]})")
            i += 1
        else:
            decisiones.append(f"Sophia toma la derecha ({monedas[j]})")
            j -= 1

        # Turno de Mateo (Estrategia Greedy)
        if i <= j:
            if monedas[i] >= monedas[j]:
                decisiones.append(f"Mateo toma la izquierda ({monedas[i]})")
                ganancia_mateo += monedas[i]
                i += 1
            else:
                decisiones.append(f"Mateo toma la derecha ({monedas[j]})")
                ganancia_mateo += monedas[j]
                j -= 1

    return decisiones, opt[0][n-1], ganancia_mateo
```

La función `resolver(monedas)` recibe el arreglo con los valores de las monedas. Primero crea la matriz `opt` que almacena la ganancia máxima de Sophia para cada intervalo.

Luego se inicializan los casos base y se completan los intervalos por longitud creciente. Para cada intervalo se calculan las ganancias posibles si Sophia toma la moneda izquierda o la derecha, considerando en cada caso la elección Greedy de Mateo. Se almacena el mayor de los dos valores como el óptimo de dicha posición.

Finalmente, `resolver` retorna el resultado de la función `reconstruir_elecciones(opt, monedas)` que se encarga de reconstruir la secuencia de jugadas calculando el valor que tendría opt[i][j] si Sophia hubiera elegido la izquierda y deduciendo así la verdadera elección. Mientras tanto, las elecciones de Mateo se deciden siguiendo la estrategia *Greedy*. La función devuelve la lista de decisiones, la ganancia máxima de Sophia y la ganancia total de Mateo.

3.2. Complejidad

Analizamos el algoritmo completo: la función `resolver`, que construye la tabla de programación dinámica y luego invoca a `reconstruir_elecciones` sobre la tabla ya completa.

Construcción de la tabla Sea n la cantidad de monedas de la fila inicial. La matriz `opt` tiene un tamaño de $n \times n$. Como solo calculamos los casos donde el inicio del intervalo no supera al final ($i \leq j$), solo estamos llenando la mitad superior de la matriz. Esto significa que la cantidad de estados que el algoritmo procesa es, aproximadamente, la mitad de n^2, es decir, $\mathcal{O}(\frac{n^2}{2})$. Como en notación de orden, las constantes se descartan, nos queda que el algoritmo calcula $\mathcal{O}(n^2)$ estados.

El algoritmo recorre las longitudes de los intervalos desde 2 hasta n. Para cada longitud, se exploran todos los intervalos posibles. En cada estado, se realiza un número constante de comparaciones lógicas, sumas aritméticas y accesos de lectura/escritura a la matriz, resultando en operaciones de costo $\mathcal{O}(1)$. Por lo tanto, el tiempo total del algoritmo estará determinado únicamente por la cantidad de celdas que debemos llenar. Como la matriz de subproblemas tiene una dimensión de $n \times n$, el costo de completar la matriz da como resultado una complejidad temporal total de $\mathcal{O}(n^2)$.

La inicialización de la matriz también requiere un tiempo de $\mathcal{O}(n^2)$, por lo que el orden del costo temporal general no cambia.

En consecuencia, la complejidad temporal de construir la tabla es:

$$\boxed{\mathcal{O}(n^2)}$$

y su complejidad espacial adicional es:

$$\boxed{\mathcal{O}(n^2)}$$

Reconstrucción Es invocada una única vez desde `resolver`, con la tabla completa. La reconstrucción de las jugadas empieza desde la solución completa en la esquina superior de la matriz, donde $i = 0$ y $j = n - 1$. A partir de ahí, el algoritmo avanza mediante un bucle que simula la partida desde el estado inicial. En el turno de Sophia, calculamos el valor que se obtendría si ella hubiera elegido la moneda izquierda y lo comparamos con el resultado real guardado en la celda $OPT[i][j]$. Si ambos números coinciden, confirmamos que tomó la izquierda, de lo contrario, deducimos que eligió la derecha. Luego de mover el índice correspondiente, simulamos el turno de Mateo eliminando la moneda más grande según su lógica *Greedy*.

Como cada una de estas comprobaciones matemáticas internas se resuelve en un tiempo constante $\mathcal{O}(1)$ y en cada iteración del bucle se descartan monedas de la mesa, el ciclo se ejecuta a lo sumo n veces hasta vaciar el tablero. Esto hace que el costo temporal de reconstruir todo el camino sea lineal, es decir, de $\mathcal{O}(n)$.

En cuanto al espacio, el único gasto de memoria es causado por la lista donde vamos anotando las `decisiones` de la partida para mostrarlas en la terminal. Como en total se retiran exactamente n monedas, esta lista tendrá como máximo una longitud de n elementos.

Por lo tanto, la complejidad temporal de la reconstrucción es:

$$\boxed{\mathcal{O}(n)}$$

y su complejidad espacial adicional es:

$$\boxed{\mathcal{O}(n)}$$

Conclusión de costo global Como `resolver` ejecuta la construcción de la tabla y luego la reconstrucción, los costos se suman: Complejidad temporal total:

$$T(n) = \mathcal{O}(n^2) + \mathcal{O}(n) = \boxed{\mathcal{O}(n^2)}$$

Complejidad espacial total:

$$S(n) = \mathcal{O}(n^2) + \mathcal{O}(n) = \boxed{\mathcal{O}(n^2)}$$

3.3. Análisis de la variabilidad de los valores de las monedas

Para analizar si la variabilidad de los valores de las monedas afecta el tiempo de ejecución del algoritmo, se realizaron pruebas con cinco tipos de secuencias, manteniendo fijo el tamaño de entrada en $n = 500$. Se consideraron secuencias con todas las monedas iguales, valores aleatorios pequeños, valores aleatorios grandes, valores crecientes y valores decrecientes.

Para cada tipo de secuencia se realizaron 15 mediciones del tiempo de ejecución de la función `resolver`. Antes de tomar las mediciones, se ejecutó el algoritmo una vez con cada conjunto de datos para reducir el efecto de la primera ejecución. En cada repetición, los cinco conjuntos se midieron en orden aleatorio, para que el orden de ejecución no favorezca ni perjudique a ninguno. Finalmente, se utilizaron las medianas de los tiempos obtenidos para comparar los resultados.

Los tiempos medidos presentaron diferencias pequeñas entre los distintos conjuntos de datos. Entre el más rápido (todas iguales con tiempo: 0,034490 segundos) y el más lento (variadas grandes con tiempo: 0,041013 segundos) hay una diferencia de aproximadamente el 19 %. Sin embargo, no se observa un patrón relacionado con el orden o la magnitud de los valores, ya que las secuencias crecientes y decrecientes tienen tiempos casi idénticos.

Por lo tanto, los resultados obtenidos no permiten concluir que el timepo de ejecución algoritmo dependa de los valores o del orden de las monedas, ya que al mantener fijo el tamaño de entrada, el algoritmo calcula la misma cantidad de subproblemas.

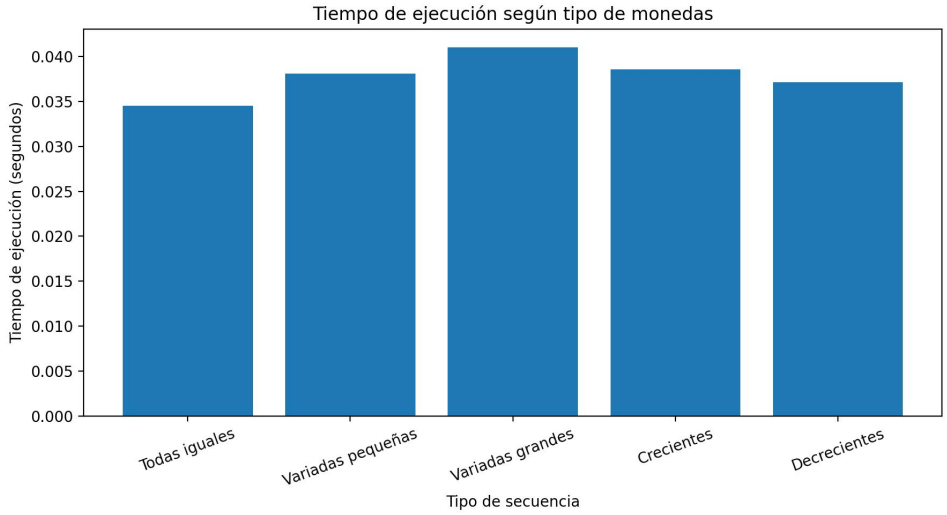


Figura 1: Comparación de las medianas de los tiempos de ejecución para distintos tipos de secuencias de monedas, con $n = 500$.

Como se observa en la Figura 1, los tiempos de ejecución son similares entre los distintos conjuntos de datos. Las diferencias que se observan son pequeñas y no siguen un patrón asociado a la estructura de la entrada, por lo que no las atribuimos al algoritmo.

Este comportamiento es compatible con la complejidad temporal teórica del algoritmo, la cual es $\mathcal{O}(n^2)$. La programación dinámica calcula la solución para todos los subintervalos posibles de la secuencia, realizando una cantidad constante de operaciones $\mathcal{O}(1)$ por cada subproblema. Por lo tanto, el número de estados depende exclusivamente de la longitud n de la entrada, y es independiente de la magnitud o el orden de los valores.

3.4. Medición de tiempos y análisis de complejidad

Para analizar experimentalmente la complejidad temporal del algoritmo, se realizaron pruebas con diferentes tamaños de entrada: $n = 100$, $n = 200$, $n = 400$, $n = 600$, $n = 800$ y $n = 1000$. Para cada tamaño se generó una secuencia de monedas con valores aleatorios entre 1 y 1000.

Para cada secuencia se midió el tiempo de ejecución de la función `resolver` completa, es decir, la construcción de la tabla de programación dinámica y la posterior reconstrucción de las decisiones. Antes de medir se ejecutó el algoritmo una vez para reducir el efecto de la primera ejecución, y luego se realizaron 15 mediciones por tamaño y utilizando la mediana de los tiempos obtenidos para reducir la influencia de posibles variaciones puntuales.

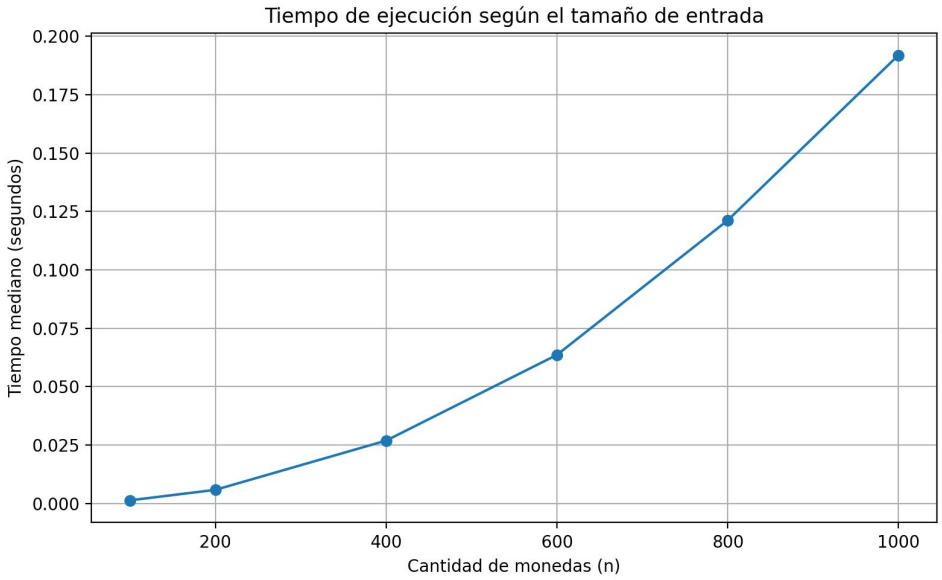


Figura 2: Mediana del tiempo de ejecución del algoritmo según la cantidad de monedas.

Los resultados muestran una clara tendencia creciente en el tiempo de ejecución a medida que aumenta el tamaño de la entrada. Si el costo es cuadrático, al duplicar n el tiempo debería multiplicarse aproximadamente por 4. Al duplicar la cantidad de monedas de 400 a 800, el tiempo pasa de aproximadamente 0,026860 s a 0,121211 s, es decir se multiplica por aproximadamente 4,5.

La complejidad temporal teórica del algoritmo es $\mathcal{O}(n^2)$, dado que la cantidad de subproblemas (intervalos $[i, j]$) que se deben resolver es proporcional a $\frac{n^2}{2}$. Para cada uno de estos subproblemas, el algoritmo realiza un número constante de operaciones $\mathcal{O}(1)$ al comparar las opciones disponibles.

Como se observa en la Figura 2, los experimentos realizados muestran que el tiempo de ejecución crece de manera compatible con la complejidad cuadrática esperada a lo largo de todos los tamaños de entrada evaluados. En conclusión, la evidencia experimental obtenida es consistente con el comportamiento $\mathcal{O}(n^2)$ deducido de la estructura de la solución por programación dinámica.

4. Conclusiones

Se formuló una solución de programación dinámica para maximizar la ganancia de Sophia frente a un oponente que aplica una estrategia greedy determinista. La recurrencia considera las dos elecciones posibles de Sophia y anticipa la respuesta de Mateo en cada caso.

La demostración por inducción permite establecer que los valores almacenados en la matriz representan la ganancia máxima que Sophia puede obtener para cada intervalo de monedas.

La implementación calcula la matriz en tiempo $\mathcal{O}(n^2)$ y espacio $\mathcal{O}(n^2)$, y reconstruye la secuencia de decisiones en tiempo adicional $\mathcal{O}(n)$.

Los experimentos realizados muestran que el tiempo de ejecución crece de manera compatible con la complejidad cuadrática esperada. También se observó que, para un mismo tamaño de entrada, las distintas distribuciones de valores de las monedas presentan tiempos de ejecución similares.

En conclusión, la comparación de estrategias muestra que una elección localmente equivalente puede tener consecuencias futuras diferentes, y que la programación dinámica puede aprovechar esas consecuencias para mejorar la ganancia de Sophia.